

ui.knob 全面详细阐述

`ui.knob` 是 NiceGUI 框架中基于 Quasar QKnob 组件实现的旋钮式数字输入组件，通过鼠标拖拽或触摸滑动实现数值调节，支持自定义颜色、尺寸、数值范围等配置，适用于音量控制、亮度调节、参数阈值设置等需要直观、连续数值输入的场景。以下从核心特性、使用方法、参数配置、API 详情等方面展开说明。

一、核心特性

- 直观交互体验**：采用旋钮拖拽交互，支持鼠标和触摸操作，数值调节连续且直观，符合用户对物理旋钮的操作习惯。
- 灵活数值控制**：可设置数值范围（`min/max`）、调节步长（`step`），默认支持 0.0 到 1.0 的浮点数调节，可扩展至任意数值区间。
- 高度样式自定义**：支持分别配置旋钮主体颜色、中心颜色、轨道颜色，支持自定义尺寸（标准尺寸或 CSS 单位），适配不同界面风格。
- 数值可视化**：通过 `show_value` 参数控制是否显示当前数值，无需额外组件即可直观反馈调节结果。
- 组件联动能力**：支持与其他组件（如滑块、标签）进行数值绑定、启用状态绑定、可见性绑定，实现复杂交互逻辑。
- 版本兼容特性**：2.16.0+ 版本支持 `html_id` 属性，3.0.0+ 支持绑定参数的 `strict` 校验，适配框架新版本特性。

二、基础使用方法

1. 最简示例：基础旋钮

快速创建带数值显示的旋钮，默认数值范围 0.0-1.0，步长 0.01：

```
from nicegui import ui

# 基础旋钮：初始值 0.3，显示当前数值
ui.knob(value=0.3, show_value=True)

ui.run()
```

2. 自定义颜色与尺寸

配置旋钮主体、中心、轨道的颜色，指定自定义尺寸（支持标准尺寸或 CSS 单位）：

```
from nicegui import ui

# 自定义颜色+标准尺寸（lg：大号）
ui.knob(
    value=0.5,
    color='orange-500',    # 旋钮主体颜色（Tailwind 颜色）
    center_color='white',  # 中心区域颜色
    track_color='grey-200', # 轨道颜色
    size='lg',             # 标准尺寸（xs/sm/md/lg/xl）
    show_value=True
```

```

)

# 自定义 CSS 尺寸 (150px 直径)
ui.knob(
    value=0.7,
    color='teal-600',
    size='150px', # CSS 单位尺寸
    show_value=True
)

ui.run()

```

3. 调整数值范围与步长

通过 `min` / `max` 扩展数值区间，`step` 控制调节精度，适配整数或小数调节场景：

```

from nicegui import ui

# 整数调节: 0-100 范围, 步长 1 (适用于音量、亮度等整数参数)
ui.knob(
    value=50,
    min=0,
    max=100,
    step=1,
    color='red-500',
    show_value=True,
    label='音量控制'
)

# 高精度小数调节: 0-10 范围, 步长 0.1 (适用于精度参数)
ui.knob(
    value=3.5,
    min=0,
    max=10,
    step=0.1,
    color='blue-500',
    show_value=True,
    label='精度调节'
)

ui.run()

```

4. 组件联动：旋钮与标签 / 滑块同步

将旋钮与标签、滑块绑定，实现数值实时同步，适用于多组件控制同一参数的场景：

```

from nicegui import ui

# 旋钮与标签绑定 (实时显示数值)
knob = ui.knob(value=0.4, show_value=False)
value_label = ui.label(f'当前值: {knob.value:.2f}').bind_text_from(
    knob, 'value', forward=lambda x: f'当前值: {x:.2f}'
)

```

```
)

# 旋钮与滑块双向绑定（相互同步数值）
slider = ui.slider(value=0.4, min=0, max=1, step=0.01)
knob.bind_value(slider)
slider.bind_value(knob)

ui.run()
```

5. 嵌套图标增强视觉识别

在旋钮中心嵌套图标，提升组件功能辨识度（如音量、亮度控制场景）：

```
from nicegui import ui

# 中心嵌套音量图标
with ui.knob(
    value=0.6,
    color='purple-500',
    center_color='purple-100',
    show_value=True
) as knob:
    ui.icon('volume_up', color='purple-700') # 嵌套图标（NiceGUI 内置图标）

# 中心嵌套亮度图标
with ui.knob(
    value=0.3,
    color='yellow-500',
    center_color='yellow-100',
    show_value=True
):
    ui.icon('brightness_medium', color='yellow-700')

ui.run()
```

三、关键参数说明

- `value`：类型为 `float`，旋钮初始数值，默认值为 `0.0`。
- `min`：类型为 `float`，允许的最小值，默认值为 `0.0`。
- `max`：类型为 `float`，允许的最大值，默认值为 `1.0`。
- `step`：类型为 `float`，数值调节步长，默认值为 `0.01`（支持整数或小数步长）。
- `color`：类型为 `str` | `None`，旋钮主体颜色，支持 Quasar、Tailwind 或 CSS 颜色格式，默认值为 `"primary"`（框架主题色）。
- `center_color`：类型为 `str` | `None`，旋钮中心区域颜色，示例值如 `primary`、`teal-10`，无默认值（继承框架默认样式）。
- `track_color`：类型为 `str` | `None`，旋钮轨道颜色，示例值如 `primary`、`teal-10`，无默认值（继承框架默认样式）。
- `size`：类型为 `str`，旋钮尺寸，支持标准尺寸名称（xs/sm/md/lg/xl）或 CSS 单位（如 `16px`、`2rem`），

无默认值（自适应界面）。

- `show_value`: 类型为 `bool`, 是否显示当前数值文本, 无默认值（需显式设置 `True/False`）。
- `on_change`: 类型为 `Callable[[ValueChangeEventArguments], Any] | Callable[[], Any]`, 数值变化时触发的回调函数, 事件对象 `e` 包含 `value` 属性（当前数值）, 默认值为 `None`。

四、高级功能

1. 动态修改属性（颜色、范围、步长）

通过组件方法动态更新旋钮的颜色、数值范围、步长等属性，适配场景变化：

```
from nicegui import ui

knob = ui.knob(
    value=0.5,
    color='green-500',
    min=0,
    max=1,
    step=0.01,
    show_value=True
)

# 动态修改主体颜色
ui.button('切换为红色', on_click=lambda: knob.props({'color': 'red-500'}))

# 动态修改数值范围和步长（切换为 0-100 整数调节）
ui.button('切换整数模式', on_click=lambda: (
    knob.set_property('min', 0),
    knob.set_property('max', 100),
    knob.set_property('step', 1),
    knob.set_value(50)
))

ui.run()
```

2. 启用 / 禁用与显示 / 隐藏控制

通过 `set_enabled()`、`set_visibility()` 或绑定开关组件，控制旋钮的可操作状态和显示状态：

```
from nicegui import ui

knob = ui.knob(value=0.3, show_value=True, color='indigo-500')

# 开关控制启用/禁用
enable_switch = ui.switch(label='启用旋钮', value=True)
enable_switch.bind_value_to(knob, 'enabled')

# 开关控制显示/隐藏
show_switch = ui.switch(label='显示旋钮', value=True)
show_switch.bind_value_to(knob, 'visible')

ui.run()
```

3. 监听数值变化触发业务逻辑

通过 `on_change` 回调函数，在数值变化时执行自定义业务逻辑（如参数更新、设备控制）：

```
from nicegui import ui

def on_knob_change(e):
    # 数值变化时弹出通知（示例：模拟音量调节）
    ui.notify(f'音量已调整至: {e.value:.0%}')
```

音量控制旋钮（0-1 映射为 0%-100%）

```
ui.knob(
    value=0.7,
    min=0,
    max=1,
    step=0.01,
    color='orange-500',
    show_value=True,
    on_change=on_knob_change
)

ui.run()
```

五、API 详情补充

1. 核心属性

属性名	类型	说明	
<code>classes</code>	<code>str</code>	组件 HTML 类名，支持 Tailwind/Quasar 类（如设置边距、阴影）	
<code>client</code>	<code>Client</code>	组件所属的客户端实例	

属性名	类型	说明	
<code>enabled</code>	<code>BindableProperty</code>	是否启用组件（可绑定，支持动态切换），默认值为 <code>True</code>	
<code>html_id</code>	<code>`str`</code>	<code>None`</code>	组件 HTML DOM ID（版本 2.16.0+），用于手动定位 DOM 元素
<code>props</code>	<code>Props[Self]</code>	组件 Quasar 原生属性（如颜色、尺寸相关配置）	
<code>style</code>	<code>style[Self]</code>	组件内联 CSS 样式（如自定义定位、边框）	
<code>value</code>	<code>BindableProperty</code>	当前数值（可绑定，支持动态同步），默认值为 <code>0.0</code>	
<code>visible</code>	<code>BindableProperty</code>	组件是否可见（可绑定，支持动态切换），默认值为 <code>True</code>	

2. 常用方法

方法名	说明	参数
<code>bind_enabled(target)</code>	双向绑定组件启用状态到目标对象（如开关）	<code>target</code> ：目标组件； <code>target_name</code> ：目标属性名（默认 <code>enabled</code> ）
<code>bind_value(target)</code>	双向绑定组件数值到目标对象（如滑块、标签）	<code>target</code> ：目标组件； <code>target_name</code> ：目标属性名（默认 <code>value</code> ）
<code>bind_visibility(target)</code>	双向绑定组件可见性到目标对象	<code>target</code> ：目标组件； <code>target_name</code> ：目标属性名（默认 <code>visible</code> ）
<code>set_enabled(bool)</code>	启用 / 禁用组件	<code>bool</code> ：True 启用，False 禁用
<code>set_value(value: float)</code>	动态设置旋钮数值	<code>value</code> ：新数值（需在 <code>min-max</code> 范围内）
<code>set_visibility(bool)</code>	显示 / 隐藏组件	<code>bool</code> ：True 显示，False 隐藏
<code>on_value_change(callback)</code>	绑定数值变化回调函数（等价于 <code>on_change</code> 参数）	<code>callback</code> ：接收 <code>ValueChangeEventArguments</code> 参数的函数
<code>tooltip(text: str)</code>	为组件添加悬浮提示	<code>text</code> ：提示文本（如“调节音量”）
<code>update()</code>	强制更新组件状态到客户端	无参数

3. 核心事件

事件名	说明	回调参数
<code>on_change</code>	数值变化时触发（拖拽过程中实时触发）	事件对象 <code>e</code> ，含 <code>value</code> 属性（当前数值）
<code>click</code>	点击旋钮时触发	通用事件对象

六、注意事项

- 数值范围限制：**`value` 必须在 `min-max` 范围内，超出范围时会自动被截断为 `min` 或 `max`，需注意初始值和动态设置值的合法性。
- 步长与精度：**使用小数步长（如 `0.001`）时，可能因浮点数精度导致数值显示微小偏差（如 `0.1 * 3 = 0.30000000000000004`），可通过 `forward` 函数格式化显示。
- 样式优先级：**`color`、`center_color`、`track_color` 若使用不同格式的颜色（如 Quasar 主题色和 Tailwind 颜色），以显式设置的颜色为准，冲突时需统一颜色格式。
- 嵌套内容限制：**旋钮中心仅支持嵌套小型元素（如图标、小文本），嵌套复杂组件可能导致布局错乱。
- 版本兼容性：**`html_id` 属性需 NiceGUI 2.16.0+，`strict` 参数绑定需 3.0.0+，使用时需确保框架版本符合要求。

七、应用场景

- 多媒体控制：音量调节、播放进度控制、亮度调节等场景，旋钮交互符合用户直觉。
- 参数配置：设备参数阈值设置（如传感器灵敏度、报警阈值）、系统参数调节（如字体大小、透明度）。
- 数据可视化控制：图表缩放比例、数据筛选范围调节，配合图表组件实现实时联动。
- 游戏控制：游戏内角色属性调节、视角灵敏度控制，提供沉浸式操作体验。
- 智能家居控制：远程控制灯光亮度、空调温度、窗帘开合程度等，旋钮形式贴近物理设备操作。

`ui.knob` 凭借直观的交互体验、灵活的样式配置和完善的联动能力，成为 NiceGUI 中处理连续数值输入的核心组件之一，适配从简单参数调节到复杂设备控制的各类场景，兼顾易用性与视觉一致性。